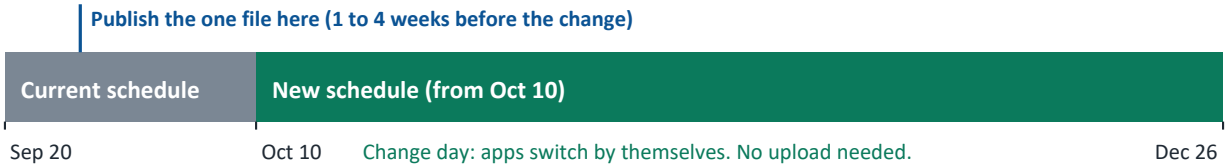
How to publish a GTFS service change in advance

A simple guide for Barrie Transit. The examples use the real files for the October 10, 2026 change.

> **The idea in one sentence**
>
> Put the **current schedule** and the **new schedule** in **one** GTFS file, give each schedule its own dates, and publish the file **1 to 4 weeks** before the change. On the change day, Google Maps, Transit and all other apps switch to the new schedule by themselves. Nobody has to upload anything on the day.



One file holds both schedules. Each schedule has its own start and end date.

Why one file?

Some apps (Google Maps and Transit) can take a file with **only** the new schedule and keep the old one until the change day. Other systems read only the file that is online now. For those, a new-schedule-only file removes all current service until the change day. One file with both schedules works for **everyone**. This is what GTFS.org recommends: *"One GTFS dataset should contain current and upcoming service."*

The steps

Step 1. Export both schedules

Export the current schedule and the new schedule from the scheduling system. If the system can export several schedule periods into one GTFS file, use that. If not, export two files and combine them (steps 2 to 5), or use a merge tool such as OneBusAway GTFS Merge.

Make sure that the two schedules do **not** use the same `service_id`, `trip_id`, `shape_id` or `block_id` values. (For Oct 10, all IDs were different, so this was already correct.)

Step 2. Give each schedule its own dates (calendar.txt)

The current schedule **ends the day before** the change. The new schedule **starts on** the change day. No gap, no overlap.

calendar.txt

```
service_id,monday,tuesday,wednesday,thursday,friday,saturday,sunday,start_date,end_date
b25cd391-1734-48dc-b428-74ef89e38a2a,1,1,1,1,1,0,0,20260920,20261009   <- current weekday, ends Oct 9
bdffc066-4aee-4313-bc58-5303d1fd4ce9,0,0,0,0,0,1,0,20260920,20261009   <- current Saturday, ends Oct 9
802d793b-b00d-4889-a2a0-0fe36c5dc988,0,0,0,0,0,0,1,20260920,20261009   <- current Sunday, ends Oct 9
569ca90d-ae31-44bc-8216-83ab4ba1f2e5,1,1,1,1,1,0,0,20261010,20261226   <- new weekday, starts Oct 10
1b35495d-aca9-4880-9c03-ea0125f330bf,0,0,0,0,0,1,0,20261010,20261226   <- new Saturday, starts Oct 10
772cd25b-0e9f-4696-90a0-4adcc6f025f3,0,0,0,0,0,0,1,20261010,20261226   <- new Sunday, starts Oct 10
```

Step 3. Put each holiday on the schedule it falls in (calendar_dates.txt)

A holiday after the change day must use the **new** schedule's `service_id`. If it uses the old one, it does nothing, and the buses show as running on the holiday.

calendar_dates.txt

```
service_id,date,exception_type
569ca90d-ae31-44bc-8216-83ab4ba1f2e5,20261012,2   <- Thanksgiving: no weekday service
569ca90d-ae31-44bc-8216-83ab4ba1f2e5,20261225,2   <- Christmas: no weekday service
1b35495d-aca9-4880-9c03-ea0125f330bf,20261226,2   <- Boxing Day: no Saturday service
```

Step 4. Combine the other files

File	What to do
trips.txt, stop_times.txt, shapes.txt	Keep the rows of **both** schedules. Each trip links to its schedule through its service_id.
routes.txt	Keep every route used by either schedule. A route that ends (for example 15A and 15B) stays in the file. It has no trips after the change, so apps stop showing it.
stops.txt	Keep every stop used by either schedule, one row for each stop. Use the **newest** stop data, including the Allandale Transit Terminal parent station and platforms.
agency.txt	Keep one copy.

Step 5. Set the feed dates (feed_info.txt)

`feed_start_date` is the first day of the **current** schedule. It must be on or before the day you publish. `feed_end_date` is the last day of the **new** schedule.

feed_info.txt (selected columns)

```
feed_publisher_name,...,feed_start_date,feed_end_date,feed_version,...
Barrie Transit,...,20260920,20261226,20260920+20261010-merged,...
```

Step 6. Check the file, then publish

- Upload the file to the MobilityData GTFS Validator (**gtfs-validator.mobilitydata.org**). Fix all errors. If you see `trip_coverage_not_active_for_next7_days` or `future_feed`, the current schedule is missing.
- Publish the file **1 to 4 weeks** before the change day.
- After the change day, you can remove the old schedule from the file in the next update.

Quick checklist

- ☐ One file has the current schedule **and** the new schedule.
- ☐ Current schedule ends the day before the change. New schedule starts on the change day.
- ☐ No service_id, trip_id, shape_id or block_id is used in both schedules.
- ☐ Holidays use the service_id of the schedule they fall in.
- ☐ feed_start_date is on or before today. feed_end_date is the last day of the new schedule.
- ☐ stops.txt is the newest version (Allandale parent station and platforms are there).
- ☐ The validator shows 0 errors.
- ☐ Published 1 to 4 weeks before the change day.

How early to publish

Source	Guidance
GTFS.org	The file should cover at least the next 7 days, and if possible the next 30 days.
Google	At least 2 days before. “A week ahead is ideal.”
Transit app	At least 7 days before the new schedule starts.

Sources

- GTFS.org, Schedule Best Practices: gtfs.org/documentation/schedule/schedule-best-practices/
- Google Transit Partners Help: support.google.com/transitpartners/answer/9047609 and /answer/10106355
- Transit, Guidelines for Producing GTFS Static Data: resources.transitapp.com/article/333-stoptimes-txt
- MobilityData GTFS Validator: gtfs-validator.mobilitydata.org
- OneBusAway GTFS Merge: developer.onebusaway.org/modules/onebusaway-gtfs-modules/current/onebusaway-gtfs-merge-cli.html

A complete example file (google_transit_merged_example.zip) is in the same folder as this guide. It combines the Sep 20 and Oct 10 schedules and passes the validator with 0 errors.